Journal of Computational Information Systems 8: 4 (2012) 1359–1371
Available at http://www.Jofcis.com

Training Feed-forward Neural Networks Using the Gradient Descent Method with the Optimal Stepsize

Liang GONG [1,*], Chengliang LIU [1], Yanming LI [1], Fuqing YUAN [2]

[1] *State Key Laboratory of Mechanical System and Vibration, Shanghai Jiao Tong University, Shanghai 200240, China*

[2] *Lab of Operation, Maintenance and Acoustics, Lulea University of Technology, Sweden*

Abstract

The most widely used algorithm for training multiplayer feedforward networks, Error BackPropagation (EBP), is an iterative gradient descend algorithm by nature. Variable stepsize is the key to fast convergence of BP networks. A new optimal stepsize algorithm is proposed for accelerating the training process. It modifies the objective function to reduce the computational complexity of the Jacobin and consequently that of Hessian matrices, and hereby directly computes the optimal iterative stepsize. The improved backpropagation algorithm helps alleviating the problem of slow convergence and oscillations. The analysis indicates that the backpropagation with optimal stepsize (BPOS) is more efficient when treating large-scale samples. The numerical experiment results on pattern recognition and function approximation problems show that the proposed algorithm possesses the features of fast convergence and less intensive computational complexity.

Keywords: BP Algorithm; Optimal Stepsize; Fast Convergence; Hessian Matrix Computation; Feedforward Neural Networks

1 Introduction

Multilayer feedforward neural networks have been the preferred neural network architectures for the solution of classification and function approximation problems due to their outstanding learning and generalization abilities. Error back propagation (EBP) is now the most used training algorithm for feed forward artificial neural networks (FFNNs). In 1986 D.E.Rumehart originated the error back propagation algorithm[1], and subsequently it is extensively employed in training the neural networks. The standard BP is an iterative gradient descend algorithm by nature. It searches on the error surface along the direction of the gradient in order to minimize the objective function of the network. Although BP training has proved to be efficient in many applications, it uses a constant stepsize, and its convergence tends to be very slow[2,3].

*Corresponding author.
Email address: gongliang_mi@sjtu.edu.cn (Liang GONG).

1553–9105 / Copyright © 2012 Binary Information Press
February 2012

Many approaches have been taken to speed up the network training process. Generally the techniques include the momentum[4,5], variable stepsize[4−9] and stochastic learning[10,11]. It has been widely know that the training convergence is determined by optimal choice of the step size rather than that of the steepest descent direction[12], so among them the backpropagation with variable stepsize (BPVS) are most widely investigated since the suitable training stepsize will significantly improve the rate of convergence[2]. Some general-purpose optimization algorithms[13,14,15] also enlighten the development of advanced BP training algorithms. However, all of these approaches lead only to slight improvement since it is difficult to find the optimal momentum factor and stepsize for the weight adjustment along the steepest descent direction and for dampening oscillations. In order to achieve a higher rate of convergence and avoid osciulating, a new searching technique for optimal stepsize has been incorporated into standard BP algorithm to form a BP variant. To obtain the optimal stepsize, the proposed algorithm directly computes the Jacobin and Hessian matrices via modifying the objective function to reduce their computational complexity. This technique breaks the taboo of regarding the optimal stepsize is just available in theory and provides a practical calculating method.

This paper is organized as follows. Section 1 gives a brief background description on the standard BP algorithm and the optimal training stepsize. In section 2, a modified objective function is introduced to reduce the computational complexity of the proposed algorithm and the BPOS (BP with Optimal Stepsize) algorithm is proposed. The comparative results between Levenberg-Marquardt BP (LMBP) and BPOS are reported in section 3. In section 4, conclusions are presented.

Nomenclature

BPOS — BP with Optimal Stepsize

b — Output signal of the hidden-layer neuron

d_n — The training sample value corresponding to the nth output

e_n, $\hat{e}_n$ — Error of the nth output

$E(W), \hat{E}(W)$ — Network output error vector

$f(\cdot)$ — The neuron transfer function

F — The objective function

$g(W), \hat{g}(W)$ — Gradient of the objective function

$G(W), \hat{G}(W)$ — Hessian matrix of the objective function

H —Total number of the hidden-layer neurons

I —Total number of the input neurons

$J(W), \hat{J}(W)$ — Jacobian matrix of the objective function

O — Total number of the output neurons

P — Total sum of network training samples

$\hat{s}(\kappa)$ —The searching unit vector

$S(W), \hat{S}(W)$ — Residual matrix of the objective function

u — The appointed neuron input

w — The network connection weight

W — The network weight vector

X — The network input vector

Y — The network output vector

Roman letters:

$\alpha, \gamma, \xi, \varphi$ — The variables used in numerical experiment

ε_1 —The network convergence index

ε_2 — The network error index

η — Network training stepsize

η^* —The optimal training stepsize

θ —The neuron bias

Θ — The network bias vector

κ — The κth iteration number

K — The maximum iteration number

Ω —The total number of the weights and biases

$\nabla^2 \hat{e}_n(W)$ — Hesse matrix of the network error

Superscripts:

l — The lth training sample, $l = 1, 2, \cdots, P$

T — Transposed matrix

Subscripts:

j — The jth neuron in the output layer $j = 1, 2, \cdots, O$

k — The kth neuron in the input layer $k = 1, 2, \cdots, I$

l — The lth training sample, $l = 1, 2, \cdots, P$

m — The mth neuron in the hidden layer $m = 1, 2, \cdots, H$

n — The nth neuron in the output layer $n = 1, 2, \cdots, O$

2 Background of Error Back Propagation (EBP) Algorithm

2.1 Standard BP algorithm

The BP algorithm for multi-layer feed-forward networks is a gradient descent scheme used to minimize a least-square objective function. H.N. Robert proved that a three-layer BP network could be trained to approximate any continuous non-linear functions with arbitrary precision, so in this paper a three-layer BP network is introduced as the example for illustrating relevant algorithms. Generally speaking, a BP network contains the input layer, hidden layer(s), and output layer. The neurons in the same layer are not interlinked while the neurons in adjacent layers are fully connected with weight W and bias Θ. For a given input X and output $Y = F(X, W, \Theta)$,

the network error is reduced to a preset value by continuously adjusting W.

The standard BP algorithm [1] defines the objective function (performance index) as

$$F(X,W,\Theta)=\frac{1}{2}\sum_{l=1}^{P}\sum_{n=1}^{O}\left(y_n^l-d_n^l\right)^2=\frac{1}{2}E^TE \tag{1}$$

where $(W,\Theta)=[W_{1,1}\cdots W_{k,m}\cdots W_{I,H}\quad W_{I+1,1}\cdots W_{I+j,m}\cdots W_{I+O,H}\quad \theta_1\cdots\theta_m\cdots\theta_H\quad \theta_{H+1}\cdots \theta_{H+j}\cdots\ \theta_{H+O}]$ is the weight and bias vector. And the error signal can be defined as

$$E=[e_{11}\cdots e_{O1}\ e_{12}\cdots e_{O2}\ \cdots\ e_{1P}\cdots e_{OP}]^T \tag{2}$$

where $e_{nl}=y_n^l-d_n^l$.

Then, we can obtain the gradient of the objective function

$$\nabla F(X,W,\Theta)=J^TE \tag{3}$$

where, the Jacobian matrix of the objective function is

$$J=\begin{bmatrix}
\frac{\partial e_{11}}{\partial w_{1,1}}\cdots\frac{\partial e_{11}}{\partial w_{k,m}}\cdots\frac{\partial e_{11}}{\partial w_{I,H}} & \frac{\partial e_{11}}{\partial w_{I+1,1}}\cdots\frac{\partial e_{11}}{\partial w_{I+i,m}}\cdots\frac{\partial e_{11}}{\partial w_{I+O,H}} & \frac{\partial e_{11}}{\partial \theta_1}\cdots\frac{\partial e_{11}}{\partial \theta_H} & \frac{\partial e_{11}}{\partial \theta_{H+1}}\cdots\frac{\partial e_{11}}{\partial \theta_{H+O}}\\
\frac{\partial e_{21}}{\partial w_{1,1}}\cdots\frac{\partial e_{21}}{\partial w_{k,m}}\cdots\frac{\partial e_{21}}{\partial w_{I,H}} & \frac{\partial e_{21}}{\partial w_{I+1,1}}\cdots\frac{\partial e_{21}}{\partial w_{I+i,m}}\cdots\frac{\partial e_{21}}{\partial w_{I+O,H}} & \frac{\partial e_{21}}{\partial \theta_1}\cdots\frac{\partial e_{21}}{\partial \theta_H} & \frac{\partial e_{21}}{\partial \theta_{H+1}}\cdots\frac{\partial e_{21}}{\partial \theta_{H+O}}\\
\vdots\quad\vdots\quad\vdots\quad\vdots\quad\vdots & \vdots\quad\vdots\quad\vdots\quad\vdots\quad\vdots & \vdots\quad\vdots\quad\vdots & \vdots\quad\vdots\quad\vdots\\
\frac{\partial e_{O1}}{\partial w_{1,1}}\cdots\frac{\partial e_{O1}}{\partial w_{k,m}}\cdots\frac{\partial e_{O1}}{\partial w_{I,H}} & \frac{\partial e_{O1}}{\partial w_{I+1,1}}\cdots\frac{\partial e_{O1}}{\partial w_{I+i,m}}\cdots\frac{\partial e_{O1}}{\partial w_{I+O,H}} & \frac{\partial e_{O1}}{\partial \theta_1}\cdots\frac{\partial e_{O1}}{\partial \theta_H} & \frac{\partial e_{O1}}{\partial \theta_{H+1}}\cdots\frac{\partial e_{O1}}{\partial \theta_{H+O}}\\
\vdots\quad\vdots\quad\vdots\quad\vdots\quad\vdots & \vdots\quad\vdots\quad\vdots\quad\vdots\quad\vdots & \vdots\quad\vdots\quad\vdots & \vdots\quad\vdots\quad\vdots\\
\frac{\partial e_{1p}}{\partial w_{1,1}}\cdots\frac{\partial e_{1p}}{\partial w_{k,m}}\cdots\frac{\partial e_{1p}}{\partial w_{I,H}} & \frac{\partial e_{1p}}{\partial w_{I+1,1}}\cdots\frac{\partial e_{1p}}{\partial w_{I+i,m}}\cdots\frac{\partial e_{1p}}{\partial w_{I+O,H}} & \frac{\partial e_{1p}}{\partial \theta_1}\cdots\frac{\partial e_{1p}}{\partial \theta_H} & \frac{\partial e_{1p}}{\partial \theta_{H+1}}\cdots\frac{\partial e_{1p}}{\partial \theta_{H+O}}\\
\frac{\partial e_{2p}}{\partial w_{1,1}}\cdots\frac{\partial e_{2p}}{\partial w_{k,m}}\cdots\frac{\partial e_{2p}}{\partial w_{I,H}} & \frac{\partial e_{2p}}{\partial w_{I+1,1}}\cdots\frac{\partial e_{2p}}{\partial w_{I+i,m}}\cdots\frac{\partial e_{2p}}{\partial w_{I+O,H}} & \frac{\partial e_{2p}}{\partial \theta_1}\cdots\frac{\partial e_{2p}}{\partial \theta_H} & \frac{\partial e_{2p}}{\partial \theta_{H+1}}\cdots\frac{\partial e_{2p}}{\partial \theta_{H+O}}\\
\vdots\quad\vdots\quad\vdots\quad\vdots\quad\vdots & \vdots\quad\vdots\quad\vdots\quad\vdots\quad\vdots & \vdots\quad\vdots\quad\vdots & \vdots\quad\vdots\quad\vdots\\
\frac{\partial e_{Op}}{\partial w_{1,1}}\cdots\frac{\partial e_{Op}}{\partial w_{k,m}}\cdots\frac{\partial e_{Op}}{\partial w_{I,H}} & \frac{\partial e_{Op}}{\partial w_{I+1,1}}\cdots\frac{\partial e_{Op}}{\partial w_{I+i,m}}\cdots\frac{\partial e_{Op}}{\partial w_{I+O,H}} & \frac{\partial e_{Op}}{\partial \theta_1}\cdots\frac{\partial e_{Op}}{\partial \theta_H} & \frac{\partial e_{Op}}{\partial \theta_{H+1}}\cdots\frac{\partial e_{Op}}{\partial \theta_{H+O}}
\end{bmatrix} \tag{4}$$

In most cases, the neuron node takes a simple nonlinear function such as Sigmoid function $f(x)=\frac{1}{1+e^{-x}}$, and then the error backpropagation process can be described as following:

For the jth neuron in the output layer, the neuron input and output are

$$u_j^l=\sum_{m=1}^{H}w_{I+j,m}\cdot b_m^l+\theta_{H+j}^l \text{ and } y_n^l=f(u_j^l) \tag{5}$$

For the mth neuron in the hidden layer, the neuron input and output are

$$u_m^l=\sum_{k=1}^{I}w_{k,m}\cdot x_k^l+\theta_m^l \text{ and } b_m^l=f(u_m^l) \tag{6}$$

All the first order partial derivative in the Jacobian matrix can be written as

$$\left.\frac{\partial e_{n,l}}{\partial w_{I+j,m}}\right|_{n\neq j}=2\left(y_n^l-d_n^l\right)\cdot b_m^l,\quad \left.\frac{\partial e_{n,l}}{\partial w_{I+j,m}}\right|_{n=j}=0 \tag{7}$$

$$\frac{\partial e_{n,l}}{\partial w_{k,m}} = 2\left(y_n^l - d_n^l\right) \cdot w_{n,m} b_m^l \cdot \left(1 - b_m^l\right) \cdot x_k^l \tag{8}$$

$$\left.\frac{\partial e_{n,l}}{\partial \theta_{H+j,m}}\right|_{n \neq j} = 2\left(y_n^l - d_n^l\right) \cdot b_m^l, \quad \left.\frac{\partial e_{n,l}}{\partial \theta_{H+j,m}}\right|_{n=j} = 0 \tag{9}$$

$$\frac{\partial e_{n,l}}{\partial \theta_{k,m}} = 2\left(y_n^l - d_n^l\right) \cdot w_{n,m} b_m^l \cdot \left(1 - b_m^l\right) \cdot x_k^l \tag{10}$$

According to Ref. [1], the gradient descent method can be employed to modify the weight vector, that is,

$$\Delta W(\kappa) = -\eta \cdot \nabla F(X, W, \Theta) \tag{11}$$

where $\kappa = 1, 2, 3, \cdots, K$ is the iteration number of the weight vector and η is the iteration stepsize whose recommended value is 0.1 to 0.4.

Since similar rules can be applied to the bias modification, we will just take into consideration the weight update in the following sections.

2.2 Basic theory of the optimal training stepsize η^*

Training the ANNs with the optimal stepsize will improve the training speed, avoid oscillations, and help escaping from the local minima. The optimal training stepsize can be concluded as following.

Let $g(W) = \nabla F(X, W) = J^T E$ and $G(W) = J(W)^T \cdot J(W) + \sum_{n=1}^{O} \sum_{l=1}^{P} e_{n,l}(W(\kappa)) \nabla^2 e_{n,l}(W(\kappa))$ be the gradient and the Hesse matrix of the objective function. According to the Taylor expansion, the quadratic form of the objective function may be written as follows

$$F(W) = F(W(\kappa)) + g(W(\kappa))^T (W - W(\kappa)) + \frac{1}{2}(W - W(\kappa)) \times G(W(\kappa)) \times (W - W(\kappa))^T \tag{12}$$

$S(W(\kappa))$ is the residual function at $W = W(\kappa)$

$$S(W(\kappa)) = \sum_{n=1}^{O} \sum_{l=1}^{P} e_{n,l}(W(\kappa)) \nabla^2 e_{n,l}(W(\kappa)) \tag{13}$$

In order to minimize the objective function, the gradient descent method searches along the steepest descent direction on the error surface. The searching unit vector can be define as

$$\hat{s}(\kappa) = -\frac{g(W(\kappa))}{\|g(W(\kappa))\|} \tag{14}$$

and the newly updated weight vector

$$W(\kappa + 1) = W(\kappa) + \eta(\kappa) \cdot \hat{s}(\kappa) \tag{15}$$

Substituting (14) into (12) gives

$$F(W(\kappa + 1)) = F(W(\kappa)) + \eta(\kappa) \cdot g(W(\kappa)) \cdot \hat{s}(\kappa) + \frac{1}{2}\eta(\kappa)^2 \cdot \hat{s}(\kappa)^T \cdot G(W(\kappa)) \cdot \hat{s}(\kappa) \tag{16}$$

Differentiating Eq. (15) with respect to the stepsize η and letting the derivative be zero will obtain

$$dF(W(\kappa+1))/d\eta(\kappa) = g(W(\kappa))^T \cdot \hat{s}(\kappa) + \eta(\kappa)^2 \cdot \hat{s}(\kappa)^T \cdot G(W(\kappa)) \cdot \hat{s}(\kappa) = 0 \tag{17}$$

Substituting (13) into (17) will get optimal stepsize

$$\eta * (\kappa) = \frac{g(W(\kappa))^T}{\hat{s}(\kappa)^T G(W(\kappa))\hat{s}(\kappa)} = \frac{\|g(W(\kappa))\|^2}{g(W(\kappa))^T \cdot G(W(\kappa)) \cdot g(W(\kappa))} \tag{18}$$

Unfortunately the computation of the Hesse matrix $G(W(\kappa))$ is never a simple thing, which severely limits the implementation of the optimal stepsize. So in the following section we will investigate the solution to the fast and easy computation of Hesse matrix and propose the BPOS algorithm.

3 Backpropagation with Optimal Stepsize (BPOS)

In this section, modifications to the standard BP will be firstly introduced in 3.1. And the Hesse matrix computation is described in 3.2 and in 3.3 several computational tricks are provided.

3.1 Objective function modification to the standard BP

Assume that the objective function be modified as the one given below, and note that the continuity requirements for BP algorithm are still preserved[17,18].

$$F(W,\Theta) = \frac{1}{2}\sum_{n=1}^{O}\left[\sum_{l=1}^{P}\left(y_n^l - d_n^l\right)^2\right]^2 = \frac{1}{2}\hat{E}^T\hat{E} \tag{19}$$

And the gradient can be written as

$$\hat{g} = \nabla F(W,\Theta) = \hat{J}^T\hat{E} \tag{20}$$

where $\hat{E} = [\hat{e}_1\ \hat{e}_1\ \cdots\ \hat{e}_n\ \cdots\ \hat{e}_O]$; $\hat{e}_n = \sum_{l=1}^{P}\left(y_n^l - d_n^l\right)^2$; $n = 1, 2, \cdots, O$.

Correspondingly the Jacobian matrix of the error term is

$$J = \begin{bmatrix} \frac{\partial\hat{e}_1}{\partial w_{1,1}} \cdots \frac{\partial\hat{e}_1}{\partial w_{k,m}} \cdots \frac{\partial\hat{e}_1}{\partial w_{I,H}} & \frac{\partial\hat{e}_1}{\partial w_{I+1,1}} \cdots \frac{\partial\hat{e}_1}{\partial w_{I+j,m}} \cdots \frac{\partial\hat{e}_1}{\partial w_{I+O,H}} & \frac{\partial\hat{e}_1}{\partial\theta_1} \cdots \frac{\partial\hat{e}_1}{\partial\theta_H} & \frac{\partial\hat{e}_1}{\partial\theta_{H+1}} \cdots \frac{\partial\hat{e}_1}{\partial\theta_{H+O}} \\ \frac{\partial\hat{e}_2}{\partial w_{1,1}} \cdots \frac{\partial\hat{e}_2}{\partial w_{k,m}} \cdots \frac{\partial\hat{e}_2}{\partial w_{I,H}} & \frac{\partial\hat{e}_2}{\partial w_{I+1,1}} \cdots \frac{\partial\hat{e}_2}{\partial w_{I+j,m}} \cdots \frac{\partial\hat{e}_2}{\partial w_{I+O,H}} & \frac{\partial\hat{e}_2}{\partial\theta_1} \cdots \frac{\partial\hat{e}_2}{\partial\theta_H} & \frac{\partial\hat{e}_2}{\partial\theta_{H+1}} \cdots \frac{\partial\hat{e}_2}{\partial\theta_{H+O}} \\ \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots & \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots & \vdots \quad \vdots \quad \vdots & \vdots \quad \vdots \quad \vdots \\ \frac{\partial\hat{e}_n}{\partial w_{1,1}} \cdots \frac{\partial\hat{e}_n}{\partial w_{k,m}} \cdots \frac{\partial\hat{e}_n}{\partial w_{I,H}} & \frac{\partial\hat{e}_n}{\partial w_{I+1,1}} \cdots \frac{\partial\hat{e}_n}{\partial w_{I+j,m}} \cdots \frac{\partial\hat{e}_n}{\partial w_{I+O,H}} & \frac{\partial\hat{e}_n}{\partial\theta_1} \cdots \frac{\partial\hat{e}_n}{\partial\theta_H} & \frac{\partial\hat{e}_n}{\partial\theta_{H+1}} \cdots \frac{\partial\hat{e}_n}{\partial\theta_{H+O}} \\ \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots & \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots & \vdots \quad \vdots \quad \vdots & \vdots \quad \vdots \quad \vdots \\ \frac{\partial\hat{e}_O}{\partial w_{1,1}} \cdots \frac{\partial\hat{e}_O}{\partial w_{k,m}} \cdots \frac{\partial\hat{e}_O}{\partial w_{I,H}} & \frac{\partial\hat{e}_O}{\partial w_{I+1,1}} \cdots \frac{\partial\hat{e}_O}{\partial w_{I+j,m}} \cdots \frac{\partial\hat{e}_O}{\partial w_{I+O,H}} & \frac{\partial\hat{e}_O}{\partial\theta_1} \cdots \frac{\partial\hat{e}_O}{\partial\theta_H} & \frac{\partial\hat{e}_O}{\partial\theta_{H+1}} \cdots \frac{\partial\hat{e}_O}{\partial\theta_{H+O}} \end{bmatrix} \tag{21}$$

$$\left.\frac{\partial\hat{e}_n}{\partial w_{I+j,m}}\right|_{n\neq j} = 2\sum_{l=1}^{P}\left(y_n^l - d_n^l\right)\cdot b_m^l, \quad \left.\frac{\partial\hat{e}_n}{\partial w_{I+j,m}}\right|_{n=j} = 0 \tag{22}$$

$$\frac{\partial \hat{e}_n}{\partial \omega_{k,m}} = 2\sum_{l=1}^{P}\left(y_n^l - d_n^l\right)\cdot w_{n,m}b_m^l\cdot\left(1-b_m^l\right)\cdot x_k^l \tag{23}$$

$$\left.\frac{\partial \hat{e}_n}{\partial \theta_{H+j,m}}\right|_{n\neq j} = 2\sum_{l=1}^{P}\left(y_n^l - d_n^l\right)\cdot b_m^l,\quad \left.\frac{\partial \hat{e}_n}{\partial \theta_{H+j,m}}\right|_{n=j} = 0 \tag{24}$$

$$\frac{\partial \hat{e}_n}{\partial \theta_{k,m}} = 2\sum_{l=1}^{P}\left(y_n^l - d_n^l\right)\cdot w_{n,m}b_m^l\cdot\left(1-b_m^l\right)\cdot x_k^l \tag{25}$$

Based on this modification, we can also obtain the new Hesse matrix and the residual function

$$\hat{G}(W) = \hat{J}(W(\kappa))^T\cdot\hat{J}(W(\kappa)) + \hat{S}(W(\kappa)) \tag{26}$$

$$\text{where } \hat{S}(W(\kappa)) = \sum_{n=1}^{P}\hat{e}_n(W(\kappa))\cdot\nabla^2\hat{e}_n(W(\kappa)) \tag{27}$$

Assume that the network have I inputs, O outputs and H hidden neurons. The total number of the weights and biases should be

$$\Omega = I\cdot H + H\cdot O + H + O \tag{28}$$

For the standard BP network the Jacobian matrix holds the size of $(P\cdot O)\times\Omega$ and the Hesse matrix may be computed via J^TJ (One Ω-by-$(P\cdot O)$ matrix multiplies one $(P\cdot O)$-by-Ω matrix) and $S(W(\kappa))$ ($P\cdot O$ times of scalar multiples of Ω-by-Ω matrix).

By contrast, the modified objective makes the training less computationally intensive and occupies less memory space. Its Jacobian matrix has the size of $O\times\Omega$. And the $\hat{J}^T\hat{J}$ can be obtained by multiplying one Ω-by-O matrix with O-by-Ω one. Computing $\hat{S}(W(\kappa))$ also just requires O times of scalar multiples of Ω-by-Ω matrix. And it is especially effective when the sample size is very large, i.e., P is a large number.

3.2 Hesse matrix profile and its computation

First, a perspective on the Hesse matrix is provided with emphasis on the intrinsic structure of $\hat{S}(W)$.

The Hesse matrix has two primary components $\hat{J}(W)$ and $\hat{S}(W)$. The former is already in hand when calculating the gradient. The later, $\hat{S}(W)$, can be divided into two parts, in which $\nabla^2\hat{e}_n(W)$ is the second-order partial derivative matrix of the nth error function. Notice that it is the Hesse matrix of residual signal $\hat{e}_n(W)$ rather than the Hesse matrix of the objective function.

When $\hat{e}_n(W)$ is a strongly non-linear function, i.e. $\nabla^2\hat{e}_n(W)$ is a large number, $\hat{S}(W)$ will also be rather large and should be calculated accurately. It is common to treat such problem by approximating $\hat{S}(W)$ with the first order information of Hesse matrix. For example, the QuasiNewton method utilizes BFGS formula to substitute the secant approximation for $\hat{S}(W)$. And the Gauss-Newton method totally ignored the $\hat{S}(W)$ term. Hence all of these methods can not behave a satisfactory precision in network training. Aiming at improving the computation accuracy, a fast calculation method of $\hat{S}(W)$ is proposed below.

Consider the Hesse matrix of the network error $\nabla^2 \hat{e}_n(W)$

$$\nabla e_n^2(W) = \begin{bmatrix} \frac{\partial^2 \hat{e}_n}{\partial w_{1,1}^2} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial w_{1,1}\partial w_{k,m}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial w_{1,1}\partial w_{I,H}} & \frac{\partial^2 \hat{e}_n}{\partial w_{1,1}\partial w_{I+1,1}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial w_{1,1}\partial w_{I+j,M}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial w_{1,1}\partial w_{I+O,H}} & \frac{\partial^2 \hat{e}_n}{\partial w_{1,1}\partial \theta_1} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial w_{1,1}\partial \theta_H} & \frac{\partial^2 \hat{e}_n}{\partial w_{1,1}\partial \theta_{H+1}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial w_{1,1}\partial \theta_{H+O}} \\ \frac{\partial^2 \hat{e}_n}{\partial w_{1,2}\partial w_{1,1}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial w_{1,2}\partial w_{k,m}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial w_{1,2}\partial w_{I,H}} & \frac{\partial^2 \hat{e}_n}{\partial w_{1,2}\partial w_{I+1,1}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial w_{1,2}\partial w_{I+j,M}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial w_{1,2}\partial w_{I+O,H}} & \frac{\partial^2 \hat{e}_n}{\partial w_{1,2}\partial \theta_1} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial w_{1,2}\partial \theta_H} & \frac{\partial^2 \hat{e}_n}{\partial w_{1,2}\partial \theta_{H+1}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial w_{1,2}\partial \theta_{H+O}} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \frac{\partial^2 \hat{e}_n}{\partial \theta_1\partial w_{1,1}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial \theta_1\partial w_{k,m}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial \theta_1\partial w_{I,H}} & \frac{\partial^2 \hat{e}_n}{\partial \theta_1\partial w_{I+1,1}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial \theta_1\partial w_{I+j,M}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial \theta_1\partial w_{I+O,H}} & \frac{\partial^2 \hat{e}_n}{\partial \theta_1^2} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial \theta_1\partial \theta_H} & \frac{\partial^2 \hat{e}_n}{\partial \theta_1\partial \theta_{H+1}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial \theta_1\partial \theta_{H+O}} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \frac{\partial^2 \hat{e}_n}{\partial \theta_{H+O}\partial w_{1,1}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial \theta_{H+O}\partial w_{k,m}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial \theta_{H+O}\partial w_{I,H}} & \frac{\partial^2 \hat{e}_n}{\partial \theta_{H+O}\partial w_{I+1,1}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial \theta_{H+O}\partial w_{I+j,M}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial w_{I+O,H}} & \frac{\partial^2 \hat{e}_n}{\partial \theta_{H+O}\partial \theta_1} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial \theta_{H+O}\partial \theta_H} & \frac{\partial^2 \hat{e}_n}{\partial \theta_{H+O}\partial \theta_{H+1}} & \cdots & \frac{\partial^2 \hat{e}_n}{\partial \theta_{H+O}^2} \end{bmatrix}$$

It is a symmetrical Ω-by-Ω square matrix, and it will be a sparse matrix if there exists a gigantic network structure.

$\hat{e}_n(W)$ is the nth net error value computed during evaluating the objective function gradient. So, $\hat{e}_n(W)\nabla^2\hat{e}_n(W)$ and its cumulative sum, $\hat{S}(W)$, are also Ω-order real symmetrical square matrices.

3.3 Several computational countermeasures

3.3.1 Symbolic computation for expression differentiation

Symbolic computation performs exact (as opposed to approximate) calculations with a rich variety of mathematical objects including the expressions and expression matrix. Typical computational tools include the Matlab, Mathematica, MAPLE, and Cayley. With these tools, formulas are differentiated symbolically, while automatic differentiation produces derivative functions[19]. Here, the symbolic computation combined with the numerical computation to provide more simple and accurate computational performance, in addition to the flexible computation for different network topology.

For example, when the matrix elements $\frac{\partial^2 \hat{e}_n}{\partial w_{1,1}\partial w_{k,m}}$, $\frac{\partial^2 \hat{e}_n}{\partial w_{1,m}\partial w_{k,m}}$ in the net error Hesse matrix are to be computed we need evaluate the partial derivative of Eq. (23). And the symbolic computation results are

$$\frac{\partial^2 \hat{e}_n}{\partial w_{1,1}\partial w_{k,m}} = 2\sum_{l=1}^{P} y_n^l \cdot \left(1 - y_n^l\right) \cdot w_{n,m} \cdot w_{n,1} \cdot x_k^l \cdot x_1^l \cdot \left[b_m^l \cdot \left(1 - b_m^l\right)\right]^2 \tag{29}$$

$$\begin{aligned} \frac{\partial^2 \hat{e}_n}{\partial w_{1,m}\partial w_{k,m}} =& 2\sum_{l=1}^{P} w_{n,m} b_m^l \cdot \left(1 - b_m^l\right) \cdot x_k^l \cdot x_1^l \left[\left(y_n^l - d_n^l\right) \cdot \left(1 - 2b_m^l\right)\right. \\ &\left. + y_n^l \cdot \left(1 - y_n^l\right) \cdot w_{n,m} \cdot b_m^l \cdot \left(1 - b_m^l\right)\right] \end{aligned} \tag{30}$$

In this way, the net error Hesse matrix will be obtained according to Eq. (29).

3.3.2 Vectorized parameter transfer and data re-use

The matrix manipulations are mostly built upon dot products and Gaxpy (Generalized scalar an x plus y) operations. And the matrix computation efficiency mainly depends on the length of the

vector operands, and a number of factors that pertain to the movement of data such as vector stride, the number of vector loads and stores, and the level of data re-use[20].

From the viewpoint of data structure the Jacobian and Hesse matrices, stored as alphabetic strings in the computer after finishing the symbolic computation, need to be converted into explicit values, which can be realized in two ways: interpreting the matrix element expressions and compiling the matrices into executable functions. Compiling execution outperforms interpreting execution because of its high efficiency. So the Jacobian and Hesse matrices are written as executable functions and will be called when necessary.

As we all know the main routine establishes stack spaces and jumps to transfer parameters when it calls subroutines. If the parameters cannot be transferred in an array format, the main function will call the same subfunction repeatedly and cause too much overhead time. This problem may be alleviated by the vectorized parameter transfer. Specifically speaking, the data during associating the formal and the actual parameters are given in vector format such as the error vector, the weight and bias vector and the network error vector.

Take for example the computation of Hesse matrix of the network error, where the weight and bias vector and the neuron output vector are prepared as formal parameters. They will be updated after each iteration and transferred to the subfunction as actual parameter to calculate $\nabla^2\hat{e}_n(W)$. Meanwhile the Gaxpy computation of $\hat{S}(W)$ also benefits from the parameter vectorization and data reuse. Given that the net error vector $\hat{E} = [\hat{e}_1\ \hat{e}_1\ \cdots\ \hat{e}_n\ \cdots \hat{e}_O]$ is transferred as the actual parameter, $\hat{S}(W)$ can be obtained from Eq. (27) by

$$\begin{aligned} &for(j = 1; j < O; j++) \\ &\hat{S}_{[1][1]} += \hat{e}_j \cdot \frac{\partial^2 \hat{e}_j}{\partial^2 w_{1,1}} \end{aligned} \tag{31}$$

where $\hat{S}_{[1][1]}$ denotes the element locates at the first row and the first column in the residual matrix. It loops O times within the function body instead of calling the Gaxpy function for O times. Thus the residual matrix may be computed rapidly.

3.4 The BPOS procedure

As shown in Eq. (18), it is necessary to compute the gradient and the Hesse matrix of the objective function for the computation of the optimal stepsize. The Jacobian matrix and the gradient can be easily obtained according to Eq. (20) and (21) since the modifications to the objective function will reduce their computational complexity. Then according to Eq. (26) the Hesse matrix requires not only the Jacobian matrix but also the residual matrix $\hat{S}(W)$ described in Eq. (27). In order to compute the residual matrix, the network error matrix $\nabla^2\hat{e}_n(W)$ should be firstly computed according to Eq. (29), and the process has been demonstrated with examples in &2.3.1. Performing Gaxpy computation with $\hat{E}$ and $\nabla^2\hat{e}_n(W)$ will get the result of $\hat{S}(W)$ and this process is shown in &2.3.2.

Now we can present BPOS by the scheme shown in Table I.

Table 1: The BPOS procedure

Algorithm BPOS

Step I: Set weight and bias vector (W, Θ); Set convergence limitation $\varepsilon_1$and network error index ε_2;
Step II: Assume the initial iteration k=0; Specify the maximum iteration K;
Step III: *Input*: Training set X;
Step IV:
1:$k = 1$;
2:**while** $k \leq K$ **do**
3: Calculate the gradient of the objective function $\nabla F(W, X, \Theta)$and the searching unit $\hat{s}(k)$;
4: **if** $\|\hat{s}(k)\| \leqq \varepsilon_1$**then** break;
5: **else**
6: Compute the optimal stepsize $\eta^*(k)$, and update $\triangle W(k+1) = \eta^*(k) \cdot \hat{s}(k)$;
7: **end if**;
8: **if** $F(W(k+1)) - F(W(k)) \leq \varepsilon_2$**then** break;
9: **else**
10: $k++$;
11: **end if**;
12:**end while**;
Step V:*Output*:k,$F(W, X, \Theta)$;

4 Case Study

Two benchmarking cases are explored to illustrate the computation experiences of the proposed BPOS and the effective Levenberg-Marquardt-BP. Since the standard BP with steepest descend method converges slowly, we omit its numerical result. In section 3.1 Iris dataset is used to show the pattern recognition via BPOS and in section 3.2 the 1000-Dimension extended Rosenbrock function is adopted to verify its effectiveness on function approximation with heavy computational load.

4.1 Pattern recognition problem

Iris data has been extensively used in literature for illustrating classification and clustering properties of neural networks. It involves classification of 150 iris flowers into three classes of species according to their four attributes, e.g. petal length, petal width, sepal length and sepal width[21], which poses a typical pattern recognition problem.

Given the 4×17×1 network topology and identical initial weights/bias, the proposed BPOS method has been compared with the LM-BP. Fig. 1 shows that BPOS outperforms LMBP with 1/3 LMBP training epochs.

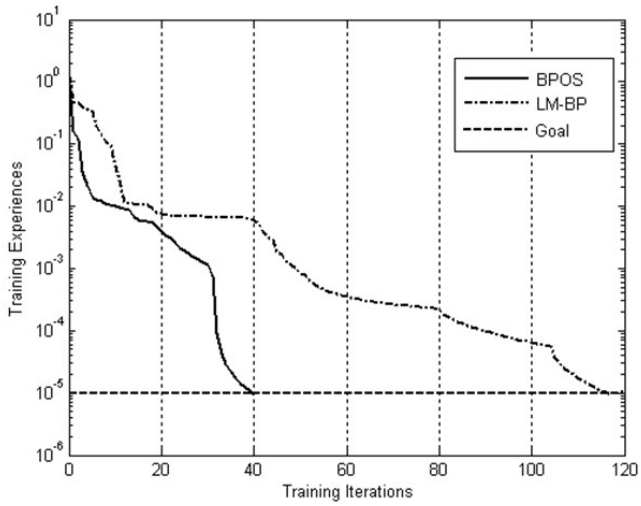

Fig. 1: Performance comparison between BPOS and LMBP on the basis of Iris dataset

4.2 Function approximation

The Rosenbrock function $R(x) = \sum_{i=1}^{N-1} [(1-x_i)^2 + 100(x_{i+1} - x_i^2)^2] \ \forall x \in \mathbf{R}^N$ is a classic test function in optimization theory[22]. It is sometimes referred to as Rosenbrock's banana function due to the shape of its contour lines. Fig. 2 illustrates a 2-Dimension Rosenbrock function has its global minimum at the point (1, 1) that lies on a plateau. In this case, some numerical solvers might take a long time to converge to it. When Rosenbrock function extends to high dimension, it is hard for training algorithms to converge into a multiple dimension narrow valley.

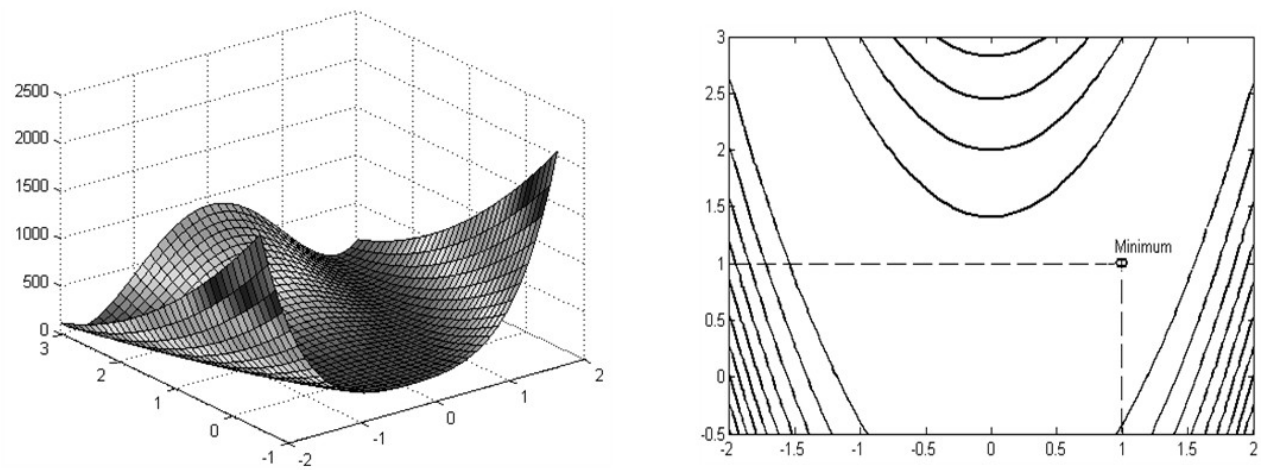

Fig. 2: 2-D rosenbrock function and its contour

As high-dimensional Rosenbrock function generates large volume of input data and 1-D output, BPOS takes advantage of these features to achieve better performance. A 500-D and 1000-D Rosenbrock function are employed to test the computational cost of BPOS on the platform of 2.0GHz Intel Core Duo CPU, 2.0G RAM, MATLAB environment and WIN7 OS. The comparison between BPOS and LMBP is given in Table II.

Numerical results demonstrate that BPOS consumes double of the computational time when the Rosenbrock function rises from 500-D to 1000-D, while LMBP costs almost 4 times of the original time. The reduction of iteration and computing time show the efficiency and effectiveness of BPOS scheme.

Table 2: Performance comparison between BPOS and LMBP when treating high dimensional Rosenbrock function

Algorithm	500-D Rosenbrock Function*		1000-D Rosenbrock Function*	
	Iteration	Time(s)	Iteration	Time(s)
BPOS	2397	1011	3481	2596
LMBP	19667	1474	44105	5560

The input dataset is discretized between [-2,2] with the interval of 0.1, and used for training in 25/50 batches.

5 Conclusion

The BPOS algorithm is proposed for training the feed-forward neural networks and proven to be effective. Optimal stepsize is obtained via exactly calculating the residual Hesse matrix. BPOS modifies the objective function to obtain less intensive computational complexity, which makes it especially suitable for the network with larger training sample number and fewer network outputs. Generally speaking, the proposed algorithm is more favorable for the nonlinear function approximation. As to convergence rate and computational time, BPOS outperforms the standard BP algorithm. Our further interests are to explore fast second-order derivative computation and to enable rapid Hesse matrix calculation on the basis of above-mentioned techniques such as symbolic computation.

Acknowledgement

This work was co-supported by the National Nature Science Foundation of China (No. 61175038), Research Project of State Key Laboratory of Mechanical System and Vibration (No. MSVM-S201103) and China Postdoctoral Science Foundation (No. 20110490724).

References

[1] D. E. Rumelhart, G. E. Hilton, AND R. J. Williams, *Learning representations by back-propagation errors.* Nature, 323 (1986), pp. 533 – 536.

[2] Y. LeCun, L. Bottou AND G. Orr, *Neural Networks: Tricks of the trade, Lecture Notes In Computer Science*, Springer, 1998.

[3] P. Smagt AND G. Hirzinger, *Why feed-forward networks are in bad shape.* Proceedings of the 8th International Conference on Artificial Neural Networks, pp. 159 – 164, Berlin, Springer, 1998.

[4] A. A. Miniani, R. D. Williams, *Acceleration of back-propagation through learning rate and momentum adaptation.* Proceedings of the International Joint Conference on Neural Networks, pp. 1676 – 1679, IEEE Press, Washington DC, 1990.

[5] D. G. Sotiropoulos, A. E. Kostopoulos AND T. N. Grapsa, *Training neural networks using two-point stepsize gradient methods.* International Conference of Numerical Analysis and Applied Mathematics, pp. 356 – 359, Greece, 2004.

[6] R. A. Jacobs, *Increased rate of convergence through learning rate adaptation*, Neural Networks, 1 (1988), pp. 295 – 307.

[7] D. Sarkar, *Methods to speed up error- back-propagation learning algorithm.* ACM Computing Surveys, 27 (1995), pp. 519 – 544.

[8] G. S. Androulakis, M. N. Vrahatis, AND G. D. Magoulas, *Efficient backpropagation training with variable stepsize*, 10 (1997), pp. 295 – 307.

[9] Y. G. Petalas AND M. N. Vrahatis, *Parallel tangent methods with variable stepsize*, Proceedings of the IEEE International Joint Conference on Neural Networks, pp. 1063 – 1066, Budapest, Hungary, IEEE Press, 2004.

[10] A. Salvetti. AND B. M. Wilamowski, *Introducing Stochastic Process within the Backpropagation Algorithm for Improved Convergence*, Proc. Artificial Neural Networks in Engineering, pp.205 – 210, St. Louis Missouri, IEEE Press, 1994.

[11] G. D. Magoulas, V. P. Plagianakos, AND M. N. Vrahatis. *Adaptive stepsize algorithms for on-line training of neural networks*, Nonlinear Analysis, 47 (2001), pp. 3425 – 3430.

[12] Y. Narushima, AND T. Wakamatsu, *Extended Barzilai-Borwein method for unconstrained minimization problems*, Pacific Journal of Optimization, 6 (2008), pp. 591 – 613.

[13] Y. Xiong, W. Wu, H. F. Lu, C. Zhang, Convergence of online gradient method for pi-sigma neural networks, Journal of Computational Information Systems, 3 (2007), 2345 – 2352.

[14] Y. Yuan, *A new stepsize for the steepest descent method*, Journal of Comp. Math. 24 (2006), pp. 149 – 156.

[15] W. J. Cheng, H. Q. Li, X. E. Ruan, *Modified quasi-Newton algorithm for training large-scale feedforward neural networks and its application*, Journal of Computational Information Systems, 7 (2011), 3047 – 3053.

[16] H. N. Robert, *Theory of the back propagation neural network*, International Joint Conference on Neural Networks, pp. 583 – 604, Washington D. C., IEEE Press, 1989.

[17] B. M. Wilamowski, O. Kaynak, AND S. Iplikci, An algorithm for fast convergence in training neural networks, Proceedings of the International Joint Conference on Neural Networks, pp. 1778 – 1782, Washington D. C., IEEE Press, 2001.

[18] J. Fan AND Y. Yuan. *On the convergence of the Levenberg-Marquardt method without nosingularity assumption.* Computing, 74 (2005), pp. 23 – 39.

[19] A. Griewank, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 19th in Frontiers in Appl. Math. Philadelphia, pp. 123 – 135, PA, SIAM, 2000.

[20] G. H. Golub AND C. F. *Loan Matrix Computations*, Johns Hopkins University Press, Baltimore, MD, 1996.

[21] UCI Machine learning repository, *http://www.ics.uci.edu/∼mlearn/MLRepository.html.* Retrieved 2010 – 09.

[22] J. J. More, B. S. Garbow AND K. E. Hillstrom, *Testing unconstrained optimization software*, ACM Transactions on Mathematical Software, 7 (1981), pp. 17 – 41.